

OpenRewrite + AI Agent

Hybrid Migration –

Conversation Log

User

I've recently implemented JDK 25 upgrade and Spring boot 4.x upgrade of one of my microservices in my project using OpenRewrite plugin + Devin AI tool

I asked Devin coding agent to work with my repo by identifying appropriate version of the OpenRewrite plugin for the recipe names that i have it upfront in the prompt for my repo upgrade.

As expected it was able to resolve the combination of dependency versions and got the plugin perform two step upgrade - dryRun phase and run phase.

Later I had the Devin Coding agent compare the exercise with a situation if it's pitched the same outcome i.e. upgrade JDK 25 and Spring boot 4.x upgrade without using OpenRewrite plugin (maybe provide some information like release notes of JDK 25 / spring boot 4.x release.

The prediction report stated it would definitely benefit to have a hybrid approach using OpenRewrite plugin than doing it without. The reasoning was mainly around OpenRewrite actually modifying files as command output rather than Devin coding tool do it by the usual edit file tool calls with appropriate code replacement snippets. Also the OpenRewrite is a one pass deterministic output as opposed to Devin work which is basically LLM non deterministic in nature especially if this activity is repeated for array of 100+ code repositories.

I'm not sure if the comparison result is correct in general sense of cost and predictability.

What's your take on the overall topic. Be very thorough in analysing this. Absolutely don't have any bias, don't assume the report by Devin to be right or wrong. Apply your own knowledge and

have your own research. Then only produce your answer.

Assistant

Short version: Devin's core conclusion — hybrid (OpenRewrite-first, AI-agent-for-the-gaps) beats a pure AI-agent rewrite for large-scale JDK/Spring Boot migrations — lines up with the actual state of the art. But the report's *reasoning* is right for the wrong emphasis in a couple of places, and it likely oversold "definitely" where the honest answer is "usually, with real exceptions." Here's the breakdown.

Where Devin's reasoning holds up

- **Determinism isn't a nice-to-have at 100+ repos, it's the whole point.** A recipe edits the AST/LST and prints it back unchanged except where it had to change — same input, same output, every time. A recipe edits the tree, then prints it back exactly as it found it, so the diff is small, format-preserving, and reviewable. An LLM agent editing files repo-by-repo can genuinely produce 18/20 correct outcomes and 2 subtly different ones

because AI produces probabilistic outputs while code migration demands deterministic ones — and at 100+ repos, that variance is a real compliance/audit problem, not just an annoyance.

- **Cost asymmetry is real and well documented, not just Devin's opinion.** One benchmark comparison put it starkly: 30,000 tokens, 3 minutes, done, versus 61 million tokens and never finished for the pure-agent path on an equivalent transformation. This is exactly the "one-pass deterministic vs re-derived-every-time LLM" cost gap Devin flagged.
- **This is the direction the whole industry (and even fintechs) has converged on,** not just a Moderne marketing claim: deterministic, scalable automation that operates with precision across thousands of repositories is what large financial institutions need, and OpenRewrite is designed to tackle exactly that, while AI agents calling deterministic recipes no longer need to invent upgrade paths.

Where the report likely oversimplified

- **OpenRewrite only automates what someone already wrote a rule for.** An academic benchmark paper is blunt about this limit: OpenRewrite operates deterministically and can only apply transformations for which an explicit rule exists — it isn't designed to handle unforeseen challenges like a third-party library incompatible with the target version with no drop-in replacement, in which case it correctly completes its prescribed refactoring but leaves the harder problem for a human. So "hybrid beats pure-OpenRewrite" is just as true a statement as "hybrid beats pure-AI" — Devin's report framed it as AI-vs-OpenRewrite when the real answer is AI-*and*-OpenRewrite, with OpenRewrite doing the bulk and AI doing triage on what's left.
- **Recipe coverage for gaps is itself a judgment call, and one independent write-up quantifies it:** for a Spring Boot upgrade, recipe-based transformations work well and produce deterministic results for standard Java API upgrades, but for third-party libraries without recipes (e.g. Ehcache2 → Ehcache3, proprietary frameworks) OpenRewrite cannot perform automated transformations, and

build/fix loops via AI can address many of those cases but need human oversight.

Translation: your `productmaster` service's custom bits (reactive/blocking mix, gRPC glue) are exactly the class of thing recipes won't touch — that's where the AI agent earns its keep, not on the mechanical JDK/Boot bump itself.

- **The Spring Boot 4.0 recipe itself is a big, actively-evolving composite** — it chains Spring Boot 3.5, Spring Framework 7, Spring Security 7, Spring Batch 5 → 6, Hibernate, Testcontainers, and more. That's a strength (huge mechanical coverage) but also means you should verify recipe maturity/coverage against your actual repo rather than assume 100% completeness, since the recipe evolves regularly.
- **"One-pass deterministic" undersells the real workflow, which is dry-run → review diff → run → build/test loop** — you already did this two-step yourself. That review step is where cost actually creeps back in regardless of tool, and Devin's report likely didn't fully account for the human review/PR-merge overhead at 100+ repos (which Moderne's

own tooling exists specifically to reduce via portfolio dashboards, not something the "recipe vs agent" comparison alone captures).

My honest take on "cost and predictability in general"

Devin's directional call is correct: for a *known, well-trodden* migration path (JDK N → N+1, Spring Boot major version), a deterministic recipe engine will almost always beat a pure LLM-agent rewrite on cost, consistency, and auditability at scale — this isn't controversial, it's converged industry practice (Moderne, FINOS/fintech usage, and independent practitioner write-ups all say the same thing). Where I'd push back on the report's framing is: don't read it as "OpenRewrite good, AI agent bad" — read it as "use the deterministic tool for the 80-95% mechanical surface, and reserve the AI agent's judgment for the residual 5-20% (missing recipes, custom code patterns, ambiguous library swaps, build/test failures)." That's a materially different (and cheaper) allocation of AI-agent effort than "let Devin rewrite the whole repo from release notes," which

is the strawman comparison the report actually ran.

For your specific situation across `productmaster` and the two gRPC services: the JDK 25 / Boot 4 mechanical parts are squarely recipe territory; the reactive/blocking MongoDB mixing and gRPC-specific wiring you've been working through are not — that's where I'd keep the agent in the loop rather than trusting a recipe to "know" your architecture.

User

Ok, devin report also claimed the repo I was migrating would have saved 50-70% ACUs than Devin alone approach.

Now I'm not sure how did it parameterized the calculation and in general how fair or average the gain would be. Could you do a research around the gains numbers if it's the hybrid approach industry go to solution then there must have been some min max savings posted if any experiments.

Assistant

Short answer: I could not find a published, standardized "50–70% ACU/cost savings" benchmark for OpenRewrite-hybrid vs pure-AI-agent migrations anywhere in the literature. That number looks like Devin's own per-repo extrapolation, not an industry figure — and here's what the actual evidence looks like, so you can judge how much to trust it.

What real data exists

1. **FreshBrew (Google/MPI/Salesforce, arXiv 2510.04852)** — the most rigorous public benchmark I found, 228 real repos, JDK 8 → 17/21 migration, comparing a rule-based OpenRewrite baseline against 7 LLM agents:
 - Pure AI agents (best: Gemini 2.5 Flash) hit **52.3% end-to-end success** on JDK 17, dropping to **49.8%** on JDK 21 — with a wide spread across models (worst was ~10.7%). This is genuine full-repo migration including test-pass and coverage-preservation checks, not just "does it compile."
 - The **OpenRewrite baseline scored much lower on raw end-to-end success (7–**

10%) as a standalone autonomous tool – but the authors are explicit: OpenRewrite was not intended to be used as an autonomous tool, but rather as a means of saving development time. In other words, the "low score" isn't a failure, it's because OpenRewrite deliberately doesn't try to fix the remaining API-incompatibility/dependency problems a human or agent normally finishes off – that gap is exactly the hybrid seam.

- On **cost**: successful agent-driven migrations clustered mostly under ~\$1–3 per repo, with GPT-4.1 the most cost-variable and DeepSeek-V3 the cheapest/most consistent – but these are *successful* runs only; failed attempts (roughly half the time) still burn tokens without yielding a usable migration, which is the real cost tax pure-agent approaches carry at scale.
- A separate industry example (Moderne) cited a case of **30,000 tokens / 3 minutes to completion** via the deterministic path versus **61 million tokens and never finished** for an equivalent pure-agent

attempt — an extreme outlier, but directionally consistent with "recipe-first is cheaper when a recipe exists."

2. No paper or vendor page I found publishes a clean "X%–Y% savings" range for the hybrid-vs-pure-agent comparison specifically.

Moderne's marketing repeats large qualitative wins (e.g., 3,500 repos in one push, 38,000 call sites fixed with zero regressions) but doesn't quantify ACU/token savings as a percentage band you could treat as a benchmark.

So how should you read Devin's "50–70%"?

Most likely it's a *self-referential* calculation: Devin probably compared its own token/step usage on your repo *with* OpenRewrite doing the mechanical lifting versus a hypothetical/simulated full rewrite it estimated itself would have taken without it. That's not wrong to compute, but it means:

- It's **one data point from one repo**, extrapolated from Devin's own cost model — not a cross-repo benchmark with min/max/median like FreshBrew provides.

- The percentage will vary hugely by repo complexity. FreshBrew's own data shows migration difficulty (and by extension, agent cost) scales with dependency count, LOC, and test count — a repo with heavy custom reactive/gRPC code (like `productmaster`) will see the AI-agent-only path cost far *more* than 50-70% extra, because more of the diff falls outside what a recipe can automate. A trivial repo with no custom logic might see savings well below 50%, because there's barely any "agent work" left to save.
- Devin's number is plausible as an order-of-magnitude (the FreshBrew and Moderne data both point the same direction), but treat it as **directionally correct, not precisely calibrated** — there's no controlled experiment behind "50-70%" the way there is behind, say, the 52.3% JDK 17 success rate figure.

If you want a real number for your own portfolio

The cleanest way to validate this yourself: run the same repo (or a representative sample of your 100+) through both paths — OpenRewrite dry-run + Devin cleanup vs. Devin-only from release notes — and log actual ACU/token spend

for each. That gives you an empirical min/max for *your* codebase's specific mix of boilerplate vs custom reactive/gRPC code, which will be far more trustworthy than either Devin's single-repo estimate or any vendor's generic marketing number.